

Airline Yield Management & Dynamic Pricing Simulation

1. What Was Done in the Code (The Methodology)

The Python script builds a complete, simulated environment of an airline's booking system over a 90-day period. Here is the step-by-step breakdown of the logic:

- **Baseline Anchoring:** The script first reads your actual CSV data (`flight_data_DEL_BOM.csv`) to calculate the median price (₹4,780). This ensures the simulation operates on realistic financial figures rather than arbitrary numbers.
- **Initialization:** We set the constraints: a 90-day booking window and an aircraft capacity of 250 seats.
- **The Dynamic Pricing Algorithm:** For every day counting down from 90 to 1, the code calculates a new ticket price using two primary factors:
 - **Time Factor:** A linear function that slowly raises the base price as the departure date approaches.
 - **Scarcity Premium (Load Factor):** An exponential function (`load_factor 2`). As the available seats drop to zero, the price shoots up dramatically. This mimics real-world airline systems protecting the final seats.
- **Demand & Elasticity Simulation:** We couldn't just assume people would buy tickets regardless of price.
 - The script uses a **Poisson distribution** to generate a natural, slightly randomized number of people looking to buy tickets each day.
 - It then applies **Price Elasticity**. The script compares our algorithm's price against the "expected" market price. If our algorithm prices the ticket too high, the simulated demand shrinks. If it prices it low (like on Day 90), demand spikes.
- **Data Logging:** Every single day's results (price, bookings, remaining seats, revenue) are recorded into a Pandas DataFrame for analysis.

2. The Final Outcome

The execution of this code produces three main outcomes:

1. **A Complete Dataset:** A day-by-day ledger (`yield_management_simulation.csv`) showing exactly how the flight filled up, what price was offered on what day, and how much revenue was captured.
2. **Key Performance Indicators (KPIs):** The simulation outputs exact final numbers. For example, in our run, it resulted in a **96.8% Load Factor** (242 out of 250 seats sold) and generated **₹1,455,146** in total route revenue.
3. **Visual Dashboards:** Three distinct charts are generated:
 - **Pricing Curve:** Shows the price dipping early and spiking late.
 - **Booking Velocity:** Shows the inventory of seats depleting over time.
 - **Yield Accumulation:** Shows how revenue compounds as the flight fills up.

3. How This Helps an Airline (Business Value)

This project directly addresses one of the most critical challenges in the airline industry: **Revenue Maximization (Yield Management)**. Here is how this logic helps an airline:

- **Capturing Both Ends of the Market:** By lowering prices 90 days out, the airline captures highly price-sensitive *leisure travelers* who plan ahead. By hiking the price exponentially in the last 14 days, the airline captures massive profit margins from *business travelers* who book last minute and are price-insensitive.
- **Preventing "Spoilage" and "Spill":** * **Spoilage:** Flying with empty seats. The algorithm drops prices early to ensure a baseline level of seats are filled to cover operating costs.
 - **Spill:** Selling out too early at cheap prices. The exponential scarcity penalty ensures the airline doesn't sell out on Day 30 for ₹4,000 when a business traveler would gladly pay ₹12,000 on Day 2.
- **Automated Reaction to Demand:** Because the simulation incorporates elasticity, it demonstrates how an airline can use automated logic (instead of manual human intervention) to adjust to market conditions on the fly. If a sudden surge of people buys tickets on Day 60, the algorithm instantly triggers the scarcity premium to slow down bookings and raise yields.